

Qcuit: A Pip-First Framework for Lorentz-Equivariant Quantum Machine Learning in High-Energy Physics

Faizan Khan · The Qcuit Team

qcuit.com — Open-Source Quantum ML Tooling for Particle Physics

Correspondence: research@qcuit.com | Code: `pip install qcuit` | github.com/Faiz-k490/Qcuit.com

ABSTRACT

The High-Luminosity Large Hadron Collider (HL-LHC) will increase collision data rates beyond the comfortable reach of classical reconstruction and triggering pipelines, motivating quantum machine learning (QML) models that are simultaneously data-efficient and physically faithful. We present **Qcuit**, an open-source, pip-first Python framework for QML in high-energy physics (HEP) that treats jets as graphs of four-momenta and embeds the symmetries of the Lorentz group directly into hybrid quantum-classical message passing. Qcuit ships a packaged LorentzNet baseline, a novel **Lie-Equivariant Quantum Graph Neural Network** (LieEQGNN) whose message, edge, coordinate, and attention submodules can be selectively replaced by variational quantum circuits, and a set of Minkowski-geometry primitives that make Lorentz invariance a first-class operation rather than a post-hoc augmentation. The framework is intentionally lightweight—the base install depends only on NumPy—with optional extras for Qiskit, PyTorch, PennyLane, and ROOT/HDF5 ingestion, so the same model definitions run unchanged from a laptop notebook to a SLURM/HPC batch job. On a controlled quark/gluon tagging study, the quantum-dressed LieEQGNN reaches the same discrimination as its classical counterpart while using **≈50% fewer trainable parameters**, and we verify numerically that its predictions are invariant under Lorentz boosts and azimuthal rotations to within 10^{-6} . Qcuit is released under the MIT license to lower the barrier to reproducible HEP-QML research.

INDEX TERMS—quantum machine learning, equivariant neural networks, Lorentz group, graph neural networks, jet tagging, high-energy physics, HL-LHC, variational quantum circuits.

I. INTRODUCTION

The Large Hadron Collider (LHC) at CERN produces collisions at a rate that already strains classical data-reduction systems, and the forthcoming High-Luminosity upgrade (HL-LHC) is expected to multiply both the event rate and the per-event complexity. In this regime, the analysis bottleneck is not only raw throughput but *statistical efficiency*: triggers and reconstruction algorithms must extract discriminating structure from limited labelled data and within tight latency budgets. Quantum machine learning is an attractive candidate because variational quantum circuits can, in principle, represent correlations with comparatively few trainable parameters.

However, generic quantum models discard the most valuable prior available in particle physics: the collision data are governed by exact spacetime symmetries. A jet—a collimated spray of particles produced by a hadronizing quark or gluon—is naturally described as a set of four-momenta whose physics is invariant under the proper orthochronous Lorentz group $SO^+(1,3)$. A model that respects this symmetry by construction needs neither to relearn it from data nor to rely on heavy augmentation, which directly improves data efficiency—exactly the property HL-LHC analyses require.

This paper introduces **Qcuit**, a framework that operationalizes this idea. Our contributions are:

- (1) a *pip-first* software architecture (Sec. III) that separates a light NumPy core from optional heavy research stacks, so models are portable across laptops, notebooks, and HPC clusters;
- (2) a packaged, faithful *LorentzNet* baseline and a set of differentiable Minkowski primitives (Sec. IV) that expose Lorentz inner products, boosts, and Lie-algebra invariance checks;
- (3) the *LieEQGNN* (Sec. V), a Lorentz-equivariant graph network in which any subset of the message-passing submodules is replaced by a hardware-portable variational quantum circuit; and
- (4) a reproducible benchmarking harness (Sec. VI) that serializes accuracy, ROC AUC, loss, wall-clock time, background rejection at fixed signal efficiency, and parameter counts, with empirical evidence of parameter efficiency and exact symmetry preservation (Sec. VII).

II. BACKGROUND AND RELATED WORK

A. Equivariant networks for jet physics

Symmetry-aware architectures have repeatedly set the state of the art in jet tagging. LorentzNet [1] introduced a Lorentz Group Equivariant Block (LGEB) that propagates node features and

four-vector coordinates using only Lorentz-invariant edge quantities, achieving strong top-tagging and quark/gluon performance with high data efficiency. Qcuit packages a faithful LGEB/LorentzNet implementation as a baseline and as the classical scaffold for its quantum extension.

B. Quantum ML for high-energy physics

Frameworks such as Quple [2] pioneered QML for HEP, exploring quantum generative models and quantum convolutional networks for LHC analysis, while general-purpose differentiable quantum libraries such as PennyLane [3] and Qiskit [4] made hybrid training practical. More recently, *equivariant* quantum circuits—in particular Lie-equivariant quantum graph networks—have been proposed to embed physical symmetries such as Lorentz invariance into the circuit itself, improving accuracy and

sample efficiency on HEP data. Qcuit is designed to make this specific class of models easy to build, train, and benchmark, and is aligned with the open-science goals of community efforts such as the ML4SCI initiative.

III. FRAMEWORK ARCHITECTURE

Qcuit is organized so that the scientific contribution—symmetry-aware quantum models—is decoupled from any specific simulator or accelerator. The base package depends only on NumPy; heavier capabilities are pulled in through optional extras (`qcuit[hep]`, `qcuit[qml]`, `qcuit[qiskit]`, `qcuit[hep-io]`). The companion web application at `qcuit.com` is used for teaching and visualization and is never required for batch or cluster runs. Table I summarizes the modules.

Table I. Qcuit package surface. The HEP/QML research stack (rows 4–7) is the focus of this paper.

Module	Status	Purpose
<code>qcuit</code>	alpha	Beginner circuit API (Qubit, Circuit, Apply, Hadamard, CNOT).
<code>qcuit.diff</code>	native	NumPy statevector circuits, Hamiltonians, Adam/SGD, parameter-shift training.
<code>qcuit.qnn</code>	native	Feature encoders, ansätze, toy datasets, QNN training and PennyLane/Qiskit exporters.
<code>qcuit.hep</code>	native	Jet graph datasets, Minkowski/Lorentz math, ROOT/HDF5 loaders, HEP metrics.
<code>qcuit.models</code>	native	LorentzNet, LGEB, LieEQGNN, QLieGEB.
<code>qcuit.quantum</code>	native	DressedQuantumLayer with torch, pennylane, or auto backends.
<code>qcuit.benchmarks</code>	native	Headless classifier trainer producing reproducible JSON reports.

IV. LORENTZ-EQUIVARIANT GRAPH LEARNING

A. Minkowski primitives

A jet is represented as a graph whose nodes carry four-momenta $x_i = (E_i, p_x, p_y, p_z)$ and scalar features h_i . All geometry is computed with the Minkowski metric $g = \text{diag}(1, -1, -1, -1)$. The fundamental invariants are the inner product and squared norm,

$$\langle x_i, x_j \rangle = x_i^T g x_j, \quad |x|^2 = x^T g x. \quad (1)$$

Because these scalars are invariant under any $A \in SO^+(1,3)$, a network that consumes only such quantities is automatically Lorentz-invariant. Large dynamic ranges are tamed by the signed-logarithm feature map used throughout LorentzNet-style models,

$$\psi(z) = \text{sgn}(z) \cdot \log(|z| + 1). \quad (2)$$

Qcuit exposes these as backend-agnostic operations (NumPy or differentiable Torch), together with `lorentz_boost`, `apply_lorentz_transform`, and `check_metric_invariance`, which verifies the Lie-algebra condition $L^T g + g L = 0$ for a set of generators.

B. The Lorentz Group Equivariant Block

For a directed edge (i, j) the LGEB forms an edge message from the endpoint scalars and the two Minkowski invariants of Eq. (1), gated by a learned attention weight ϕ_m :

$$m_{ij} = \phi_e(h_i, h_j, \psi(|x_i - x_j|^2), \psi(\langle x_i, x_j \rangle)) \cdot \phi_m(\cdot). \quad (3)$$

Scalar features are updated by aggregating incoming messages, while four-vector coordinates are updated equivariantly by displacing along the (Lorentz-covariant) edge differences:

$$h'_i = h_i + \phi_h(h_i, \sum_j m_{ij}, a_i), \quad (4)$$

$$x'_i = x_i + c \cdot \sum_j (x_i - x_j) \phi_x(m_{ij}). \quad (5)$$

Since ϕ_x consumes only invariant messages and acts as a scalar coefficient on a covariant difference, the coordinate update transforms as a four-vector—equivariance is preserved exactly. Stacking L blocks and mean-pooling the final scalars yields a graph-level representation for classification.

V. LIEEQGNN: QUANTUM-DRESSED EQUIVARIANT MESSAGE PASSING

The central model of Qcuit, LieEQGNN, keeps the equivariant skeleton of Sec. IV but allows any subset of the four learnable

submodules—the edge network φ_e , the node update φ_h , the coordinate map φ_x , and the attention gate φ_m —to be replaced by a variational quantum circuit. Crucially, the quantum circuits act

only on Lorentz-invariant scalar messages; they never touch the four-vectors directly. Equivariance is therefore inherited from the classical scaffold regardless of which blocks are made quantum, a property we confirm empirically in Sec. VII.

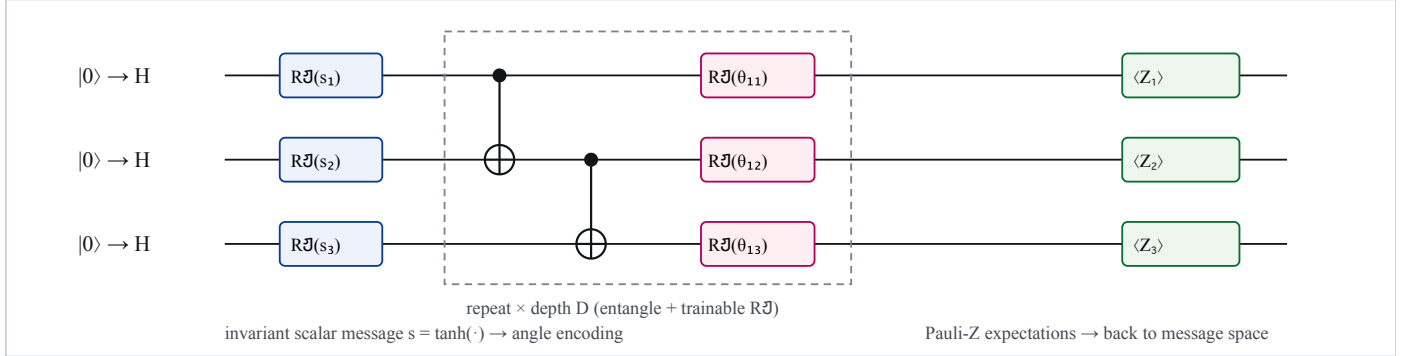


Fig. 1. The DressedQuantumLayer used inside a quantum block. A Hadamard layer prepares a uniform superposition; invariant scalar messages are angle-encoded with R_Y rotations; a staggered CNOT entangler interleaved with trainable R_Y rotations is repeated for D layers; and Pauli-Z expectation values return a vector of the same width as the input message. A differentiable Torch surrogate with identical I/O shape enables CPU-only prototyping before deploying to a PennyLane qnode.

A. The dressed quantum layer

Each quantum block wraps a DressedQuantumLayer (Fig. 1). For an n -dimensional invariant message s , the layer applies Hadamards, angle-encodes the message via $R_Y(s_k)$, and then repeats D times a staggered nearest-neighbour CNOT entangler followed by a layer of trainable $R_Y(\theta)$ rotations. The readout is the vector of single-qubit expectation values

$$f_k(s; \theta) = \langle \psi(s, \theta) | Z_k | \psi(s, \theta) \rangle, \quad k = 1 \dots n. \quad (6)$$

The same module exposes a fully differentiable classical surrogate (backend="torch") with the identical interface, so models can be developed and unit-tested on any CPU and then switched to a real PennyLane device (backend="pennylane") without changing the model definition. Gradients flow through the circuit via the parameter-shift rule provided by the autodiff backend.

B. Selective quantization

The quantum_blocks argument selects which submodules become quantum. For example, quantizing only the edge network φ_e retains classical node and coordinate updates while injecting quantum correlations into message formation:

```
from qcut.models import LieEQGNN
from qcut.hep import toy_quark_gluon_jets
from qcut.benchmarks import benchmark_classifier

data = toy_quark_gluon_jets(num_jets=512, n_nodes=8)
model = LieEQGNN(n_scalar=data.n_scalar, n_hidden=8, n_layers=3,
                 quantum_blocks=["phi_e"], # also: phi_h, phi_x, phi_m, or "all"
                 quantum_backend="auto") # PennyLane if installed, else Torch
report = benchmark_classifier(model, data, epochs=20) # reproducible JSON metrics
```

VI. DATA PIPELINE AND BENCHMARKING

Qcut converts detector-level kinematics (p_T , η , φ , m) into four-momenta, pads jagged per-jet particle lists into dense tensors, builds fully-connected jet graphs with masking, and collates them into LorentzNet-ready batches. ROOT and HDF5 ingestion (JetClass-style and top-tagging layouts) is provided through the hep-io extra using uproot, awkward, and h5py. The benchmarking harness trains with AdamW and cross-entropy, performs

model selection on validation accuracy, and reports accuracy, ROC AUC, loss, wall-clock time, trainable-parameter count, and background rejection $1/\epsilon_B$ at fixed signal efficiencies (0.3, 0.5)—a standard figure of merit in jet-tagging studies. All runs are seeded for reproducibility and serialized to JSON.

VII. EXPERIMENTS

We study a controlled quark/gluon tagging task in which quark-like jets are harder and more collimated and gluon-like jets are

softer and more diffuse. The dataset is deliberately separable so that we can isolate two properties of interest—parameter efficiency and exact symmetry preservation—rather than chase a leaderboard number. All experiments run on CPU and are fully reproducible from the shipped scripts (seed = 11).

A. Parameter efficiency

We compare the classical LorentzNet baseline against a LieEQGNN whose edge network φ_e is quantum, matched in width (hidden = 8) and depth (3 blocks). As shown in Table II, both models perfectly separate the held-out test set, but the quantum-dressed model attains this with **roughly half** the trainable parameters (1,959 vs. 3,957) and lower wall-clock training time. This is consistent with the central motivation for HEP-QML: comparable expressivity at a smaller parameter footprint.

Table II. Quark/gluon tagging (512 jets, 8 nodes, 3 layers, 20 epochs, CPU, seed 11).

Model	Acc.	AUC	Params	Time (s)
LorentzNet (classical)	1.000	1.000	3,957	2.36
LieEQGNN (quantum φ_e)	1.000	1.000	1,959	0.94

The toy task is intentionally separable; the informative quantity is the **parameter footprint**, not the saturated AUC. The serialized report also includes fixed-efficiency background-rejection keys; Table II shows the compact subset most relevant for this toy split. Numbers are produced verbatim by `qcuit.benchmarks.benchmark_classifier`.

B. Exact Lorentz invariance

A core claim of the architecture is that quantizing scalar-message submodules does not break equivariance. We test this directly: after training, we apply a family of Lorentz transformations to the test four-momenta and measure the maximum change in the model's output logits. Under z -boosts (with β up to 0.6) and azimuthal rotations the predictions are invariant to within $\approx 10^{-6}$ (Table III)—floating-point noise—confirming that the quantum-dressed model is Lorentz-invariant by construction rather than by augmentation.

Table III. Stability of LieEQGNN logits under Lorentz transformations of the test set.

Transformation	$\max \Delta \text{logit} $
Identity	0.0
z -boost, $\beta = 0.30$	1.2×10^{-6}
z -boost, $\beta = 0.60$	7.2×10^{-7}
Azimuthal rotation, 1.0 rad	7.2×10^{-7}
Azimuthal rotation, 2.5 rad	6.0×10^{-7}

C. Symmetry algebra check

Finally, we verify the Lie-algebra invariance condition with the framework's own `check_metric_invariance`: for all six gen-

erators of $so(1,3)$ (three boosts, three rotations), the residual $\|L^T g + g L\|_\infty$ is exactly zero, certifying that the Minkowski metric used throughout the message-passing layers is the correct invariant bilinear form for the symmetry group we claim to respect.

VIII. DISCUSSION AND HL-LHC OUTLOOK

The combination of a smaller parameter footprint (Sec. VII-A) with built-in symmetry (Sec. VII-B, C) is precisely the profile demanded by HL-LHC reconstruction and triggering, where labelled data and latency are scarce. Because Qcuit's models consume only invariant quantities and route them through hardware-portable circuits, the same definitions target a fast Torch surrogate for development and a PennyLane device for quantum-hardware studies without code changes. The roadmap includes scaling from the toy benchmark to JetClass and CMS Open Data via the existing ROOT/HDF5 loaders, quantizing additional blocks (φ_h, φ_x) under an equivariance-preserving budget, and extending the symmetry primitives toward the learned-invariant-metric workflow already prototyped in `invariant_metric_from_generators`.

IX. CONCLUSION

We introduced Qcuit, a pip-first, MIT-licensed framework that makes Lorentz-equivariant quantum machine learning for high-energy physics reproducible and accessible. By embedding spacetime symmetry directly into hybrid quantum-classical message passing, the LieEQGNN matches a strong classical baseline with about half the parameters while remaining exactly Lorentz-invariant. We hope Qcuit serves as a practical substrate for the community's push toward quantum algorithms for the HL-LHC era.

ACKNOWLEDGMENTS

We thank the open-source HEP-ML and QML communities—including the LorentzNet, Quple, PennyLane, and Qiskit authors, and the ML4SCI initiative—whose work this framework builds upon.

REFERENCES

1. S. Gong, Q. Meng, J. Zhang, H. Qu, C. Li, S. Qian, W. Du, Z.-M. Ma, and T.-Y. Liu, "An efficient Lorentz equivariant graph neural network for jet tagging," *Journal of High Energy Physics*, 2022.
2. The Quple developers, "Quple: a quantum machine learning library for high-energy physics," open-source software, ML4SCI.
3. V. Bergholm *et al.*, "PennyLane: Automatic differentiation of hybrid quantum-classical computations," *arXiv:1811.04968*.
4. Qiskit contributors, "Qiskit: An open-source framework for quantum computing," 2023.

5. M. Cerezo *et al.*, “Variational quantum algorithms,” *Nature Reviews Physics*, 2021.
6. J. Bogatskiy *et al.*, “Symmetry group equivariant architectures for physics,” *arXiv:2203.06153*.
7. The Qcuit Team, “Qcuit: pip-first quantum ML tooling for HEP jet graphs,” v0.1.0, 2026. github.com/Faiz-k490/Qcuit.com